

Project Context: This project is an independent implementation of a ray tracer using Ray Tracing in One Weekend as the reference. The baseline is a simple single-threaded and CPU based, it generates a scene by shooting rays from the camera and recursively tracing the interactions within the scene. Object data is stored in vectors, and pixel values are output in RGB format to a PPM file.

Local Desktop Info:

- CPU; Intel(R) Core(TM) i7-14700F
- 8 P-Cores (2 threads each), 12 E-Cores (1 thread each)
- Instruction Set Extensions: Intel® AVX2
- RAM: 32 GB
- OS: Windows 11
- Architecture: x86-64

Ray Tracer Camera & Rendering Configs:

- aspect ratio: 16/9
- max ray depth: 50
- image width: 1200
- samples per pixel: 500

Assumptions:

- Timing on local desktop, but perf is ran on CSIL computers
- Sections get their own respective timer
- Total timer wrapped around the main
- Desktop Power Mode: Balanced

Inside ../ray-tracer/src/:

```
g++ -std=c++23 -Wall -Wextra -pedantic -O3 -march=native -g main.cpp -o raytracer
```

===== Local Desktop Runtime (4 Run Average, Hand Calculated) =====

```
Initialize Time:      0.000000 s (0 ms, 0 us)
Scene Setup Time:     0.000079 s (0 ms, 79 us)
Render Time:          1069.159577 s (1069159 ms, 1069159576 us)
Write Output Time:    0.016134 s (16 ms, 16134 us)
Total Time:           1069.175790 s (1069175 ms, 1069175789 us)
Render Fraction:      99.998483% (Approximate)
```

===== Profiler Results =====

```
68.01%    [...] Sphere::hit(Ray const&, Interval, HitRecord&) const
28.89%    [...] HittableList::hit(Ray const&, Interval, HitRecord&)
0.97%     [...] Renderer::render(Camera const&, HittableList const&
```

Hypothesis:

The biggest bottleneck is the Sphere::hit() function according to perf. To contextualize, Sphere::hit contains 4 variable initializations (with one being a constructor of type Vec3, which is a custom struct with an array[3] underlying). It has 3 if-branches. It calls 1 std::sqrt, and the 5 other mathematical functions which are vector-math or inequality based underneath so this shouldn't be as expensive, and ends with assignment operations.

Call Pipeline: Per pixel → random_sampling() → recurse if ray or depth valid: [ray_color() → Sphere::hit()]

Looking one layer higher, of when Sphere::hit is called in the rendering layer, inside of the function ray_color(...). Effectively, ray_color() is updating and returning values which is why, even though they are often invoked with each other, Sphere::hit() dominates the expense. Other factors such as rays missing/absorbed, recursions ending early and such could make reduce the amount Sphere::hit() is called, but we can calculate the possible upper theoretical limit of Sphere::hit() calls (our ray tracer config upper bound is ~20.25 billion using the formula below):

Possible_Upper_Bound = max_ray-depth * samples_per_pixel * (image_width / aspect_ratio) * image_width

I do not see a clear way to optimize within Sphere::hit() itself yet, but we could optimize around it. 1) Since the work is currently sequential, but the rendering of the pixels can be parallelized, we can use parallelism on the layer that calls Sphere::hit (the render loop) with multithreading. 2) We can also use some SIMD instructions on registers on the layer below (the Vec3 class), for the vector math functions inside of Sphere::hit.

We can calculate that the upper limit speed up is 19.96 times improvement to baseline, based on Amdahl's law $\approx 1 / ((1 - 0.9999) + 0.9999/20) \approx 19.96$ times, where P is the render section taking up ~99.99% runtime and N is 20 physical processors of my desktop. Though the upper limit improvement is assumed to be optimal conditions, where P is completely parallelizable and that all processors perform the same. Though realistically, I expect the improvement to be lower, maybe 10-12 times improvement.

Implementation Info:

My first change was to implement multithreading. Using static partitioning, I initially created 28 threads based on hardware_concurrency (to get my logical processor count) and divided the image_height by the amount of threads, to get the chunk. 182.713479 s was the average render time after 4 runs, still taking up 99.99% of the total runtime. $1069.159577 / 182.713479 \approx 5.85$ times improvement, not the same as the 12-15 times improvement I predicted.

I wondered if there was still work left idling even with 28, so I decided to use some coarse benchmarks to approximate the performance of varying thread count. Within my device and within the tests ran, the preferred range is around 1.25-1.75 of the processor count. It could be because some areas of the scene are less work (like mostly background or missed rays), so with our base number the bottleneck becomes the overloaded thread. Dividing the work further distributed the work between the threads more evenly. After a certain point, the overhead of having too many threads outweighed the gains from further parallelism, causing performance to eventually degrade. Code from this point forward is staying with $28 * 1.75$ (49).

Worker	Baseline (1)	28 * 0.25 (7)	28 * 0.75 (21)	Processor (28)	28 * 1.25 (35)	28 * 1.75 (49)	28 * 2.0 (56)
Render Time	1069.159577 s	306.562377 s	200.087552 s	182.713479 s	179.394688 s	177.095872 s	181.283038 s

*Timings are average of 4 runs, varying approximately $\pm 1.5\%$ of median timing respective to each benchmark

===== Profiler Results (After Multithreading) =====

```
49.86%    [.] Sphere::hit(Ray const&, Interval, HitRecord&) const
47.90%    [.] HittableList::hit(Ray const&, Interval, HitRecord&) const
0.98%     [.] std::thread::_State_impl<std::thread::_Invoker<std::tuple<main::{lambda()#1}>>>::_M_run()
```

Before proceeding, I wanted to check that cache misses and branch mispredictions were not major bottlenecks. Based on the results below from perf, cache misses and branching mispredictions do not hamper performance as they are minimal. The one thing that does stand out is the L1 cache miss rate is much higher than the other metrics, but is not drastic. The L1 cache miss could stem from the usage of std::share_ptr to handle multiple ownership of objects within a scene.

```
4,504,765,920 branch-misses / 2,900,305,563,937 branches      ≈ 0.155 % branches missed
121,226,181 cache-misses / 1,873,156,884,231 cache-references ≈ 0.006 % caches missed
772,562,441,402 L1-dcache-load-misses / 9,172,786,905,161 L1-dcache-loads ≈ 8.422 % L1 caches missed
23,077,988 LLC-load-misses / 60,972,659,750 LLC-loads        ≈ 0.038 % L3 caches missed
```

(Left is inlined vector math code from my Vec3 class, right are expensive sections of Sphere::hit)

<pre>Samples: 15M of event 'cpu/cycles/P', 4000 Hz, Event count (approx.): 162904 Sphere::hit(Ray const&, Interval, HitRecord&) const /home/hca252/desktop/perf Percent 0.39 vmovss %xmm0,%rcx 6.92 vmovq %xmm0,%rcx5 0.00 vmovdqa %xmm0,%rcx5 0.02 float x() const { return vec[0]; } 0.02 vmovq (%rsi),%xmm9 0.20 return Vec3(a.x() - b.x(), a.y() - b.y(), a.z() - b.z()); 6.92 vmovss %xmm9,%xmm2 shr \$0x20,%rcx float z() const { return vec[2]; } 0.01 vmovshdup %xmm10,%xmm7 return Vec3(a.x() - b.x(), a.y() - b.y(), a.z() - b.z()); 0.03 vsubss %xmm9,%xmm3,%xmm3 return (vec[0] * vec[0] + vec[1] * vec[1] + vec[2] * vec[2]); 0.00 vmovss %xmm3,%xmm1 return Vec3(a.x() - b.x(), a.y() - b.y(), a.z() - b.z()); 6.94 vsubss %xmm9,%xmm2,%xmm2 float z() const { return vec[2]; } 0.01 vmovss %xmm9,%xmm8 return Vec3(a.x() - b.x(), a.y() - b.y(), a.z() - b.z()); 11.42 vmovss %xmm8,%xmm0 return (vec[0] * vec[0] + vec[1] * vec[1] + vec[2] * vec[2]); 0.00 vmovss %xmm0,%xmm6 return Vec3(a.x() - b.x(), a.y() - b.y(), a.z() - b.z()); 7.59 vsubss %xmm9,%xmm0,%xmm0 return (vec[0] * vec[0] + vec[1] * vec[1] + vec[2] * vec[2]); 0.03 vmulss %xmm7,%xmm7,%xmm4</pre>	<pre>Samples: 15M of event 'cpu/cycles/P', 4000 Hz, Event count (approx.): 162904 Percent Vec3 center_sphere = center - r.origin; auto a = r.direction.magnitude_squared(); auto b = dot_product(r.direction, center_sphere); auto c = center_sphere.magnitude_squared() - radius * radius; 0.73 vmovss %xmm4,%xmm0 6.40 vfmadd231ss %xmm5,%xmm0,%xmm4 return a.x() * b.x() + a.y() * b.y() + a.z() * b.z(); 5.00 vfmadd231ss %xmm0,%xmm0,%xmm1 return (vec[0] * vec[0] + vec[1] * vec[1] + vec[2] * vec[2]); 0.19 vfmadd132ss %xmm0,%xmm2,%xmm0 auto discriminant = b * b - a * c; if (discriminant < 0) { 0.00 vxorps %xmm2,%xmm2,%xmm2 auto c = center_sphere.magnitude_squared() - radius * radius; 9.80 vfmadd231ss %xmm3,%xmm3,%xmm0 auto discriminant = b * b - a * c; 4.19 vmulss %xmm4,%xmm0,%xmm0 4.15 vfmadd231ss %xmm1,%xmm1,%xmm0 if (discriminant < 0) { 2.54 vcmovss %xmm0,%xmm2 i ja 100 using ::sqrt; #ifndef __CORRECT_TSO_CPP_MATH_H_PROTO inline __LIBCXX_CONSTEXPR float sqrt(float __x) { return __builtin_sqrtf(__x); } 0.35 vsqrts %xmm0,%xmm0,%xmm0 //see 10 for hint on ray hit tests</pre>
--	--

Using perf annotate, I entered Sphere::hit to investigate the remaining bottleneck, revealing the expensive sections and the assembly code underneath. With the red-highlighted assembly to guide my investigation, the hottest assembly lines are within my inlined subtraction operator(...) for vec3, the vec3 dot products inside of Sphere::hit,

and the calculation for the c term in the quadratic formula calculation. Because of the abundance of v_ss instructions, this could signal that the compiler is not fully vectorizing the code.

Before beginning, I realized that my Vec3 struct consisted of three float values (96 bits total), which is too small to naturally fit with a 128 or 256 bit floating point register. If my Vec3 had been contiguous and aligned (SoA), I could comfortably process them using SIMD (xxx|yyy|zzz...) but the baseline uses AoS, so I either have to refactor or find alternatives. One of the simpler alternatives I found was to use a padding float in the struct to have a total of 128 bits in my struct to align the bits with the SIMD registers (xyzp|xyzp|xyzp...). The extra member means more memory overhead, so I will run another perf to confirm if it affects cache hits afterwards.

As mentioned earlier, I planned on using SIMD types for the Vec3 class. My approach was to use Agner Fog's Vectorclass Library to convert the individual data members in the struct into one Vec4f. Admittedly, this approach will not make the individual data members continuous and cannot fully utilize SIMD through batching the computations, which SoA could leverage but would require a major refactoring of the structure of the ray tracer. Even still, it produced a very slight boost compared to the multithreaded render times. The timings of four runs average out to: 173.145820 s, a speed up of about 6.17 times

The extra padding memory overhead accumulated, resulting in a 10 percent point increase in L1 cache misses which is non-trivial, likely cancelling out some of the benefits of the Vec4f, but overall cache misses remained low:

138,779,694 cache-misses / 2,096,604,229,946 cache-references $\approx 0.007\%$ caches missed
1,089,830,600,159 L1-dcache-load-misses / 5,763,674,834,448 L1-dcache-loads $\approx 18.912\%$ L1 caches missed

Summary:

Ultimately, I was never able to reach the 10-12 times speed up that I expected through my approaches. Through a process of using low-level tools to profile and using the inner workings of the computer to our advantage (such as out of order execution, parallelism, the compiler and its optimizer) I was able to reduce the rendering time of my semi-high resolution scene from about 1069.159577 seconds to 173.145820 seconds, a total speed up of ~6.17 times within my local device. Because of the simplicity of my approaches, the overall performance of the ray tracer has room for improvement, but the speed up is still meaningful considering that there are many advanced CPU and algorithmic optimizations to still employ in the future.

Table Summary	Average Render Runtime	Improvement (to Baseline)
Baseline	1069.159577 s	—
Multithreading (28)	182.713479 s	5.85 times
Multithreading (28 * 1.75)	177.095872 s	6.04 times
Vec3f Wrapper	173.145820 s	6.17 times.

Render (Same config, physically sized down in document)

